FACULDADE DE TECNOLOGIA PADRE DANILO DE OLIVEIRA OHL

EDUARDO GARCIA DANTAS

GEOVANA LOPES DE SOUSA

JOÃO EMANUEL DE BEZERRA BARCELOS

VINICIUS MONTEIRO DOS SANTOS

DESENVOLVIMENTO DE SOFTWARE ASSISTIDO POR IA, OBJETO DE ESTUDO: GESTÃO DA GRADE HORÁRIA DA FATEC BARUERI.

BARUERI

2026

FACULDADE DE TECNOLOGIA PADRE DANILO DE OLIVEIRA OHL

DESENVOLVIMENTO DE SOFTWARE ASSISTIDO POR IA, OBJETO DE ESTUDO: GESTÃO DA GRADE HORÁRIA DA FATEC BARUERI.

Projeto de pesquisa apresentado à banca examinadora da Faculdade de Tecnologia de Barueri como requisito parcial para obtenção do título de tecnólogo em 2027.

Orientador: Prof. Vander Ribeiro Elme.

BARUERI

2026

LISTA DE QUADROS

Quadro 1 – Subcaracterísticas da manutenibilidade segundo a ISO/IEC 25010:2023 ..... **Erro! Indicador não definido.**

LISTA DE TABELAS

Tabela 1 – Matriz metodológica de métricas de manutenibilidade (ISO/IEC 25010) ..... **Erro! Indicador não definido.**

Tabela 2 – Cronograma de execução das etapas da pesquisa**Erro! Indicador não definido.**

Tabela 3 – Orçamento previsto..... **Erro! Indicador não definido.**

LISTA DE ABREVIATURAS E SIGLAS

ABNT ..... Associação Brasileira de Normas Técnicas

API..... Application Programming Interface (Interface de Programação de Aplicações)

CI/CD ..... Continuous Integration / Continuous Delivery (Integração e Entrega Contínuas)

DAO ..... Data Access Object (Objeto de Acesso a Dados)

FATEC ..... Faculdade de Tecnologia

IA ..... Inteligência Artificial

IAG ..... Inteligência Artificial Generativa

IEC..... International Electrotechnical Commission

ISO..... International Organization for Standardization

LLM..... Large Language Model (Grande Modelo de Linguagem)

MI ..... Maintainability Index (Índice de Manutenibilidade)

MVC ..... Model-View-Controller

NBR..... Norma Brasileira

PHP..... PHP: Hypertext Preprocessor

PSR ..... PHP Standards Recommendation

SBC..... Sociedade Brasileira de Computação

SDD ..... Spec-Driven Development (Desenvolvimento Orientado por Especificações)

SOLID ..... Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion

SSE ..... Server-Sent Events

TDD..... Test-Driven Development (Desenvolvimento Guiado por Testes)

TG..... Trabalho de Graduação

V(G)..... Complexidade Ciclomática de McCabe

VPS..... Virtual Private Server

SUMÁRIO

Nenhuma entrada de sumário foi encontrada.

1 INTRODUÇÃO

1.1 Contextualização e Delimitação do Tema

O uso de ferramentas de Inteligência Artificial Generativa (IAG) e Grandes Modelos de Linguagem (LLMs) tem acelerado a geração de código-fonte, mas também apresenta desafios relacionados à qualidade e à manutenibilidade do software. Osorio et al. (2025) identificam que a redução do tempo de programação não implica necessariamente maior correção funcional, enquanto Aquino et al. (2025) investigam problemas de qualidade e débitos técnicos em códigos gerados por LLMs.

Diante desse problema, propõe-se combinar o Desenvolvimento Orientado por Especificações (Spec-Driven Development — SDD) com o Desenvolvimento Guiado por Testes (Test-Driven Development — TDD), utilizando requisitos estruturados e testes automatizados para orientar e validar o código gerado por IA (Beck, 2010). Essa abordagem será aplicada à concepção de um sistema web para gestão e atualização da grade horária acadêmica da Fatec Barueri, com foco na avaliação da manutenibilidade e da qualidade arquitetural do código e na automação de rotinas via agendamento de tarefas (Cron).

1.2 Problema de Pesquisa

Dado o contexto, coloca-se a seguinte questão norteadora: de que maneira a integração entre Especificações Executáveis (SDD) e Desenvolvimento Guiado por Testes (TDD) impacta a manutenibilidade e a qualidade arquitetural do código-fonte gerado por IA no desenvolvimento de um sistema web de grade horária acadêmica com rotinas automatizadas por Cron, sob a ótica da norma ISO/IEC 25010 e da complexidade ciclomática de McCabe?

1.3 Hipótese de Pesquisa

Hipotetiza-se que o uso conjunto de especificações estruturadas, testes automatizados prévios e intervenção humana reduz a ocorrência de problemas de qualidade e contribui para a manutenibilidade do código gerado por IA, garantindo o cumprimento das características da ISO/IEC 25010 (ISO, 2023) e o controle da complexidade ciclomática (McCabe, 1976) em limiares recomendados ($V(G) \leq 10$).

2 OBJETIVOS

2.1 Objetivo Geral

Propor, aplicar e avaliar empiricamente um fluxo de desenvolvimento de software assistido por Inteligência Artificial baseado em Spec-Driven Development e Test-Driven Development (TDD), medindo seu impacto na manutenibilidade e na qualidade arquitetural do código-fonte através do estudo de caso da construção de um sistema web para gestão de grade horária na Fatec Barueri com suporte a rotinas agendadas em segundo plano (Cron).

2.2 Objetivos Específicos

1. Elicitar e formalizar os requisitos funcionais, não funcionais e as restrições institucionais de alocação acadêmica (University Course Timetabling Problem) da Fatec Barueri sob a forma de especificações executáveis (Spec-Driven).

2. Estruturar o ciclo metodológico de TDD (Red-Green-Refactor) integrado à geração de código por IA, estabelecendo suítes de testes unitários automatizados em ambiente PHP/PHPUnit antes da geração das rotinas de produção.

3. Projetar a arquitetura do sistema web e suas rotinas de atualização periódica e sincronização agendada via Cron, fundamentada nos padrões MVC, Repository/DAO e princípios SOLID de Clean Code.

4. Formular uma matriz multidimensional de métricas de manutenibilidade baseada na ISO/IEC 25010, na métrica de complexidade ciclomática de McCabe e em análises estáticas de código (SonarQube e PSR-12).

5. Coletar e analisar os dados de qualidade e manutenibilidade durante a fase de execução (TG 2), avaliando a eficácia do método na mitigação de débitos técnicos e alucinações.

3 JUSTIFICATIVA

A elaboração deste trabalho justifica-se sob as dimensões acadêmica, tecnológica e institucional. No âmbito acadêmico e científico, existe uma lacuna relevante na literatura quanto ao uso seguro e metodologicamente orientado de LLMs na engenharia de software. Embora estudos comprovem ganhos na velocidade de codificação, a proliferação de falhas funcionais e débitos técnicos exige investigar mecanismos de contenção baseados em engenharia de requisitos rigorosa e testes automatizados.

Sob a perspectiva tecnológica, o uso de automação agendada via Cron garante a atualização eficiente e determinística dos dados da grade horária sem sobrecarregar a infraestrutura do servidor nem exigir conexões persistentes complexas, mantendo alta modularidade e facilitando testes isolados. Institucionalmente, o projeto atende a uma demanda real da Fatec Barueri, substituindo processos manuais e passíveis de choques de horários por uma solução automatizada, ágil e de fácil manutenção preventiva e corretiva.

4 REFERENCIAL TEÓRICO

4.1 Engenharia de Software e Ciclo de Desenvolvimento

A Engenharia de Software estabelece uma abordagem estruturada, disciplinada e quantificável para o desenvolvimento, operação e manutenção de sistemas computacionais. Conforme teorizado por Pressman e Maxim (2011), a engenharia de software é uma tecnologia em camadas fundamentada no compromisso organizacional com a qualidade. O processo de software incorpora atividades estruturais fundamentais — comunicação, planejamento, modelagem, construção e implantação — integradas por ações de controle de qualidade e gestão de configuração.

No contexto de aplicações web, a manutenibilidade do código-fonte figura como um atributo crítico para assegurar a longevidade do sistema. Pressman e Maxim (2011) enfatizam que a manutenção representa a fatia mais dispendiosa do ciclo de vida do software, sendo facilitada quando princípios arquiteturais sólidos — como a modularidade efetiva, o baixo acoplamento entre componentes e a alta coesão funcional — são aplicados desde as fases iniciais de modelagem.

4.2 Inteligência Artificial Generativa e Grandes Modelos de Linguagem (LLMs)

A recente introdução de modelos de linguagem de grande porte (LLMs) no ecossistema de desenvolvimento de software redefiniu os paradigmas de produtividade. Ferramentas como o GitHub Copilot atuam como assistentes virtuais capazes de sintetizar trechos completos de código a partir de rotinas prévias ou instruções em linguagem natural.

Entretanto, investigações empíricas recentes revelam um quadro complexo em relação à qualidade do código gerado por IA. Osorio et al. (2025) demonstraram em estudo empírico que, embora o uso do Copilot reduza significativamente o tempo necessário para a conclusão de tarefas de programação, ele não apresenta diferença estatisticamente significativa na

correção funcional do código (medida pela taxa de testes unitários falhos). Complementarmente, Lira et al. (2025) avaliaram a efetividade de diferentes LLMs na geração automatizada de testes unitários, evidenciando que a precisão dos modelos varia amplamente em função da estrutura dos prompts fornecidos. Por sua vez, Aquino et al. (2025) analisaram soluções geradas por modelos como GPT-4, Gemini e Llama, identificando que, apesar das taxas de acerto iniciais, o código gerado por LLMs tende a acumular um volume expressivo de débitos técnicos e problemas de manutenibilidade em análises estáticas.

4.3 Desenvolvimento Orientado por Especificações (Spec-Driven Development)

O Desenvolvimento Orientado por Especificações (Spec-Driven Development - SDD) fundamenta-se no princípio de que o comportamento do sistema deve ser formalmente descrito através de especificações claras, estruturadas e executáveis antes do início da codificação. As especificações funcionam como um contrato arquitetural que delimita o escopo da aplicação e orienta tanto os desenvolvedores humanos quanto as ferramentas de IA generativa.

No desenvolvimento assistido por IA, a especificação executável atua como um mecanismo de alinhamento de contexto (prompt contextual). Ao fornecer à IA especificações não ambíguas (por exemplo, na forma de arquivos de requisitos estruturados ou contratos OpenAPI), mitiga-se a sensibilidade do modelo a enunciados vagos, reduzindo drasticamente a ocorrência de alucinações e garantindo que o código gerado atenda com precisão às regras de negócio estabelecidas.

4.4 Desenvolvimento Guiado por Testes (Test-Driven Development - TDD)

Formulado por Kent Beck (2010), o Desenvolvimento Guiado por Testes (TDD) inverte a sequência convencional de programação ao exigir que os testes automatizados sejam escritos antes do código de produção. O processo segue o ciclo iterativo conhecido como Red-Green-Refactor: (1) escreve-se um teste que falha; (2) implementa-se o código mínimo necessário para fazer o teste passar; e (3) refatora-se o código-fonte eliminando duplicações e melhorando a estrutura arquitetural.

Beck (2010) ressalta que o objetivo do TDD é produzir 'código limpo que funciona'. Os testes automatizados funcionam como uma catraca de segurança que impede a regressão do código. No fluxo de desenvolvimento assistido por IA, o TDD desempenha o papel de validador determinístico (Human-in-the-Loop): a IA é solicitada a gerar o código apenas para satisfazer

a uma suíte de testes unitários criada ou auditada previamente pelo desenvolvedor humano, contendo imediatamente eventuais alucinações sintáticas ou lógicas do modelo de linguagem.

4.5 Qualidade de Software, ISO/IEC 25010 e Manutenibilidade

A avaliação da qualidade do produto de software fundamenta-se no modelo normativo ISO/IEC 25010 (2023), que estabelece a Manutenibilidade como uma das características centrais da qualidade. A manutenibilidade representa a eficácia e eficiência com que um produto de software pode ser modificado por seus desenvolvedores, desdobrando-se em cinco subcaracterísticas principais, sintetizadas no Quadro 1:

Subcaracterística	Definição normativa
Modularidade	Grau em que um sistema é composto por componentes discretos, de modo que alterações em um módulo causem impacto mínimo nos demais.
Reutilização	Capacidade de um ativo de software ser utilizado na construção de outros componentes.
Analisabilidade	Facilidade com que se pode avaliar o impacto de alterações ou identificar a causa raiz de falhas no código-fonte.
Modificabilidade	Grau em que o software pode ser modificado de forma eficaz sem introduzir defeitos ou degradar a qualidade existente.
Testabilidade	Facilidade com que critérios de teste podem ser estabelecidos para o sistema e executados para validar suas alterações.

Quadro 1 – Subcaracterísticas da manutenibilidade segundo a ISO/IEC 25010:2023.
Fonte: adaptado de ISO/IEC 25010:2023.

Para mensurar quantitativamente a manutenibilidade, a literatura e a prática da engenharia de software utilizam métricas consolidadas. A Complexidade Ciclomática de McCabe V(G) mede o número de caminhos estruturalmente independentes no fluxo de controle de um método. Em termos normativos e operacionais, fixa-se o limiar recomendado de $V(G) \leq 10$

por método. Métodos que ultrapassam esse valor apresentam maior probabilidade de defeitos e elevada dificuldade de manutenção.

4.6 Domínio do Problema: Alocação de Grade Horária Acadêmica Web com Processamento Agendado (Cron)

A alocação de grades horárias em instituições de ensino é categorizada na literatura de computação como o University Course Timetabling Problem. Trata-se de um problema de otimização combinatória de alta complexidade, no qual se busca distribuir turmas, disciplinas, professores e salas de aula ao longo de intervalos de tempo predeterminados.

O problema envolve o cumprimento rigoroso de restrições rígidas (hard constraints) — tais como a impossibilidade de alocar um mesmo professor ou sala em dois horários simultâneos — e o atendimento flexível a restrições de preferência (soft constraints). Na Fatec Barueri, a atualização e a consistência dos dados da grade horária são tratadas de forma automatizada por rotinas de processamento em segundo plano agendadas via Cron (Cron Job Scheduling). Essa escolha arquitetural simplifica o ambiente de implantação, reduz o consumo de recursos computacionais e isola as rotinas de verificação e cálculo de conflitos em tarefas modulares e facilmente testáveis via TDD.

5 METODOLOGIA (MATERIAIS E MÉTODOS)

5.1 Enquadramento Metodológico

A pesquisa enquadra-se como de natureza aplicada, com abordagem qualitativa e quantitativa (mista). O procedimento metodológico adotado é o Estudo de Caso Único e Aplicado, orientado pelas diretrizes de Robert K. Yin (2015) para estudos de caso nas ciências aplicadas, e pelas diretrizes específicas de Per Runeson e Martin Höst (2009) para a condução e relato de estudos de caso em Engenharia de Software. Adicionalmente, adota-se a perspectiva de triangulação metodológica em pesquisas científicas (Gil), combinando a análise documental de requisitos, a observação do processo de desenvolvimento assistido por IA e a coleta automatizada de métricas de código.

5.2 Etapas de Execução do Estudo de Caso

O estudo de caso será executado ao longo das seguintes etapas operacionais detalhadas:

1. Levantamento e Formalização de Requisitos: Mapeamento junto à coordenação da Fatec Barueri e escrita das especificações executáveis (Spec-Driven).

2. Configuração do Ecossistema de Desenvolvimento: Estruturação do ambiente PHP 8.x, servidor web, banco de dados MySQL, agendador Cron, framework de testes PHPUnit e ferramentas de análise estática de código (SonarQube e PHP CodeSniffer com regras PSR-12).

3. Ciclo de Desenvolvimento Assistido por IA (Spec-Driven + TDD): Para cada funcionalidade do sistema de grade horária e rotina do Cron, o desenvolvedor humano elaborará os testes unitários no PHPUnit. Em seguida, os testes e as especificações serão fornecidos como prompt para a IA sintetizar o código de produção (camadas MVC e Repository/DAO).

4. Análise e Coleta de Métricas: Extração automatizada das métricas de complexidade, acoplamento, coesão, cobertura de testes e dívida técnica.

5. Avaliação e Comparação: Análise estatística descritiva dos indicadores para verificar se o código gerado atendeu aos limiares de manutenibilidade estipulados.

5.3 Matriz Metodológica de Métricas de Manutenibilidade

A Tabela 1 sintetiza o protocolo rigoroso de métricas que será aplicado no TG 2 para mensurar a manutenibilidade do produto de software resultante:

Subcaracterística	Métrica / Indicador	Instrumento	Limiar adotado
Modularidade	Acoplamento eferente (Ce) e instabilidade	PhpMetrics / SonarQube	Instabilidade < 0,50
Reutilização	Reaproveitamento de componentes de serviço	SonarQube	≥ 60% de reuso
Analisabilidade	Complexidade ciclomática de McCabe V(G)	PhpMetrics / PHPMD	V(G) ≤ 10 por método
Modificabilidade	Índice de manutenibilidade e	SonarQube	MI > 85 e

	densidade de code smells		duplicação ≤ 5%
Testabilidade	Cobertura de linhas e de ramificações	PHPUnit + Xdebug	Cobertura ≥ 80%

Tabela 1 – Matriz metodológica de métricas de manutenibilidade (ISO/IEC 25010).
Fonte: elaborado pelos autores (2026).

6 CRONOGRAMA

A Tabela 2 apresenta o planejamento cronológico das etapas de pesquisa do TG 1 e desenvolvimento do TG 2:

Etapa	Período
Levantamento e formalização dos requisitos	Fev/2026 – Mar/2026
Configuração do ecossistema de desenvolvimento e rotinas Cron	Mar/2026
Ciclo Spec-Driven + TDD assistido por IA	Abr/2026 – Ago/2026
Coleta e análise das métricas de qualidade	Set/2026 – Out/2026
Redação final e defesa (TG 2)	Nov/2026

Tabela 2 – Cronograma de execução das etapas da pesquisa.
Fonte: elaborado pelos autores (2026).

7 ORÇAMENTO PREVISTO

A Tabela 3 discrimina a estimativa dos recursos financeiros e materiais para a realização da pesquisa:

Item	Quantidade	Valor unitário (R$)	Total (R$)

Hospedagem em servidor VPS	12 meses	30,00	360,00
Registro de domínio .com.br	1	40,00	40,00
Assinatura de assistente de IA	10 meses	100,00	1.000,00
SonarQube Community / PHPUnit	1	0,00	0,00
Total	**—**	**—**	**1.400,00**

Tabela 3 – Orçamento previsto.
Fonte: elaborado pelos autores (2026).

8 RESULTADOS ESPERADOS

Espera-se como resultado principal deste projeto a validação de um fluxo metodológico replicável de desenvolvimento de software assistido por IA, capaz de mitigar alucinações e conter a proliferação de débitos técnicos através do uso conjunto de especificações executáveis e TDD.

Como produto prático para o TG 2, prevê-se a entrega do sistema web de gestão de grade horária acadêmica da Fatec Barueri com atualização agendada via Cron, acompanhado de relatórios de análise estática demonstrando que o código produzido obteve 100% de conformidade com a PSR-12, complexidade ciclomática $V(G) \leq 10$ por método, cobertura de testes automatizados superior a 80% e reduzido índice de acoplamento.

REFERÊNCIAS

AQUINO, Bernardo; BRITO, Aline; TAVARES, Cleiton; BOECHAT, Danilo; BATISTELI, João Pedro. Analisando a qualidade e eficácia de códigos gerados por LLMs: um estudo com problemas da plataforma LeetCode. In: WORKSHOP SOBRE VISUALIZAÇÃO, EVOLUÇÃO E MANUTENÇÃO DE SOFTWARE (VEM), 2025. Anais [...]. [S. l.: s. n.], 2025. p. 1-12.

BECK, Kent. TDD Desenvolvimento Guiado por Testes. Porto Alegre: Bookman, 2010.

FATEC BARUERI. Manual para Elaboração de Trabalhos Acadêmico-Científicos: Projeto de Pesquisa, Artigo Científico, Monografia, Relato de Experiência e Plano de Negócio. Barueri: Faculdade de Tecnologia Padre Danilo José de Oliveira Ohl, 2025.

ISO/IEC. ISO/IEC 25010:2023: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. Genebra: ISO, 2023.

LIRA, Werney Ayala Luz; SANTOS NETO, Pedro de Alcântara dos; AVELINO, Guilherme Amaral; OSÓRIO, Luiz Fernando Mendes. Evaluating the Effectiveness and Cost-Efficiency of Large Language Models in Automated Unit Test Generation. In: SBQS 2025 - Simpósio Brasileiro de Qualidade de Software. São José dos Campos: SBC, 2025.

MCCABE, Thomas J. A Complexity Measure. IEEE Transactions on Software Engineering, v. SE-2, n. 4, p. 308-320, 1976.

OSORIO, Luiz Fernando Mendes; SANTOS NETO, Pedro de Alcântara dos; AVELINO, Guilherme Amaral; LIRA, Werney Ayala Luz. An Evaluation of the Impact of Code Generation Tools on Software Development. In: SBSI 2025 - Simpósio Brasileiro de Sistemas de Informação. Recife: SBC, 2025. p. 625-634.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. 7. ed. Porto Alegre: AMGH, 2011.

RUNESON, Per; HÖST, Martin. Guidelines for conducting and reporting case study research in software engineering. Empirical Software Engineering, v. 14, n. 2, p. 131-164, 2009.

YIN, Robert K. Estudo de Caso: Planejamento e Métodos. 5. ed. Porto Alegre: Bookman, 2015.